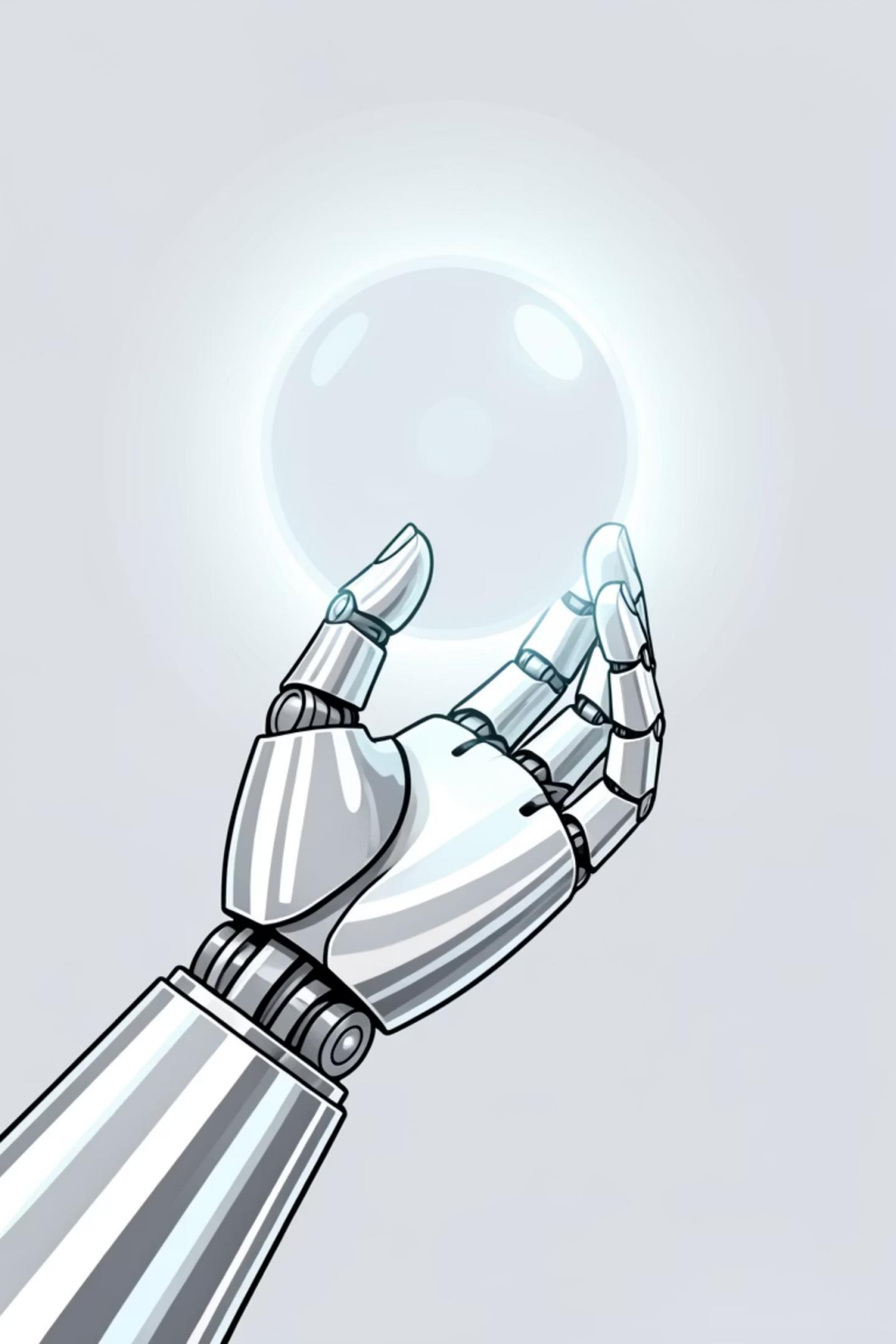




Study of Domain Randomization for Sim- to-Sim Transfer in Reinforcement Learning

Presentation of the Extension of the Robot Learning Project.

Project by Carlo Squarcia Mateo, taught at the [Politecnico di Torino](#).

Introduction to Domain Randomization (DR)

Domain Randomization (DR) is a fundamental technique that randomizes environment parameters during the training of a reinforcement learning policy.

This methodology exposes the policy to diverse dynamics in each episode, promoting the learning of **robust and invariant behaviors** that can adapt to unknown conditions.

- ❏ **Uniform Domain Randomization (UDR):** Parameters are sampled independently from uniform distributions, ensuring broad exploration of the environment variation space.



Project Objectives and Contributions

This project explores the influence of Domain Randomization (DR) on simulator-to-simulator transfer, delving not only into performance but also into the **stability and internal behavior** of trained policies.

1

Analysis of Randomized Body Parts

Identification of the most critical physical components for effective transfer.

2

Analysis of Randomization Strength

Evaluation of the impact of randomization magnitude on stability and performance.

3

Analysis of Different Architectures

Investigation of how network capacity and architecture design influence transfer and robustness.

4

Stability and Failure Analysis

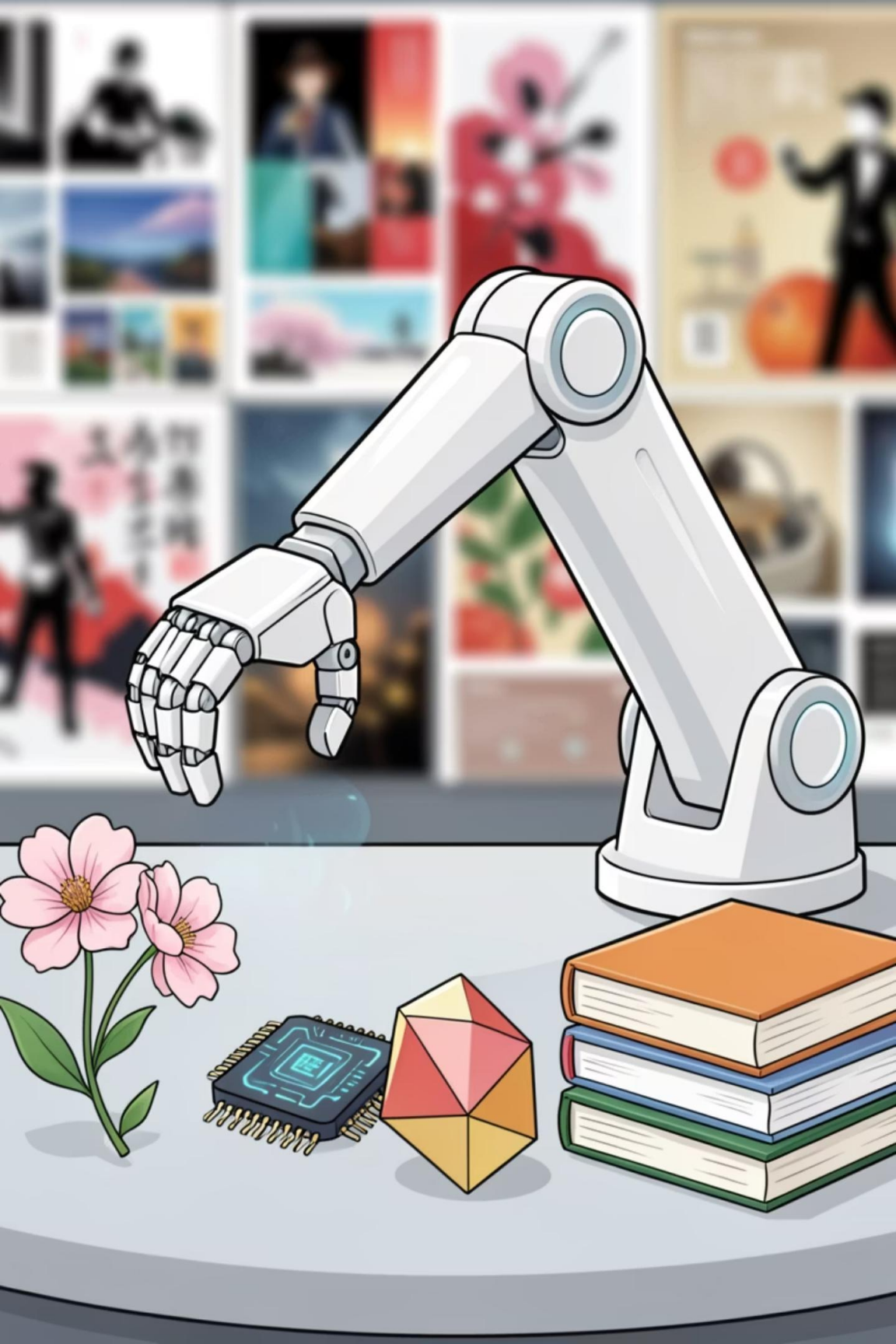
Study of policy robustness under diverse conditions.

5

Analysis of Latent Representations

Use of PCA and t-SNE to understand the policy's internal strategies.

How Does Domain Randomization Work?



Source Simulator

Randomization

Training

Target Deployment

DR allows training agents in varied environments in the source simulator so they generalize better to new target simulators without needing retraining.

Experiment 1: Which Body Parts to Randomize?

This experiment seeks to identify the **most influential physical components** in the policy transfer process.

Objective

Determine which parts of the robot's body are most important for effective simulator-to-simulator transfer.

Approach

Training policies by randomizing only one body part at a time and comparing them with full-body randomization.

Evaluation

Comparison of performance in source and target environments to measure transfer effectiveness.

Experiment 1: Key Results

The findings of this experiment reveal crucial insights about the effectiveness of domain randomization on specific parts of the robot's body.

Effect of Randomizing Individual Body Parts

No DR	1569.04 ± 5.71	835.23 ± 17.87
DR thigh (±20%)	1381.00 ± 184.36	1039.34 ± 28.70
DR leg (±20%)	1016.46 ± 117.45	708.15 ± 14.81
DR foot (±20%)	1522.51 ± 6.15	1265.99 ± 251.53
DR all (±20%)	1543.24 ± 11.28	1320.41 ± 109.04

Foot Improvement

Randomizing the **foot** significantly improves performance in the target environment, suggesting its critical role in surface interaction.



Optimal Transfer

Randomizing **all body parts** provides the best overall transfer, highlighting the benefit of comprehensive variability.



Less Effective Proximal Components

Randomization of more proximal components proved to be **less effective** in improving robustness, in contrast to distal parts.

In conclusion, **components related to contact** are fundamental to achieving robustness and successful transfer in reinforcement learning.

Experiment 2: Randomization Strength

This experiment investigates how the **magnitude of randomization** (δ) affects the performance and stability of trained policies.



Objective

Study the impact of the degree of randomization on the learning and transfer process.



Approach

A sweep of δ values (0.05, 0.10, 0.20, 0.40) was performed to evaluate their influence.



Evaluation

Performance analyzed for both foot-only randomization and full-body randomization.

Experiment 2: Key Results

The choice of randomization strength (δ) is a critical factor that directly impacts the transfer capacity and robustness of policies.

Configuration	$\pm 5\%$	$\pm 10\%$	$\pm 20\%$	$\pm 40\%$
Foot	S:1323 T:1004	S:1533 T:1155	S:1523 T:1266	S:1678 T:1105
All	S:1627 T:807	S:1099 T:884	S:1543 T:1320	S:1208 T:817

Where S = Source Return and T = Target Return.

→ Small δ : Insufficient Coverage

A low δ fails to adequately cover the dynamics of the target environment, resulting in suboptimal performance.

→ Large δ : Stability Degradation

An excessively high δ can destabilize the learning process, making policy convergence difficult.

→ Optimal δ : Balance

A $\delta = 0.20$ proves to be the best compromise, offering a balance between domain exploration and learning stability.

In conclusion, moderate domain randomization is essential to achieve effective transfer and robust policies.

Experiment 3: Policy Network Architecture

This experiment focuses on analyzing how the **capacity of the network** of the policy's neural network influences robustness and performance in the context of domain randomization.

Objective

Evaluate the impact of different network sizes on the policy's ability to generalize and maintain stability.

Approach

Network architectures with different capacities were tested: [64,64], [128,128], and [256,256].

Methodology

Multiple random seeds and a fixed DR configuration were used to ensure consistency of results.

Experiment 3: Key Results

The results of this experiment reveal a complex relationship between network capacity, performance, and stability in policy training.

Architecture	Source Return	Target Return
[64; 64]	1551 ± 81	1048 ± 238
[128; 128]	1476 ± 234	1234 ± 453
[256; 256]	1603 ± 163	1294 ± 350

Network architecture performance comparison showing source and target environment returns with standard deviations.



Improvement with Large Networks

Larger networks tend to **improve average performance** in the target environment, thanks to their greater representational capacity.



Increase in Variance

The variance between different random seeds **increases with network size**, indicating lower training stability.



Key Trade-off

There is a **trade-off** between network expressiveness and the stability of the policy training process.

In conclusion, although larger architectures may offer higher performance, they also introduce a **greater instability** that must be carefully managed for robust implementation.

Failure and Stability Analysis (Beyond Reward)

Reward alone does not fully explain robustness in robotic locomotion; therefore, we must look beyond it to understand true stability and generalization.

We delve deeper into the robot's behavior by analyzing these critical metrics:



Episode Length and Time-to-Failure

Measures how long the robot maintains stable locomotion before an episode ends, indicating overall resilience.



Minimum Torso Height

Tracks the lowest point reached by the robot's torso, serving as an indicator of how close the robot comes to falling.



Torso Angle Variability

Assesses the deviation of the robot's torso angle, reflecting its ability to maintain balance and control its posture.



Near-Fall Events

Identifies instances where the robot experiences significant instability but successfully recovers, highlighting its reactive robustness.

Stability Analysis: Key Interpretations

Analyzing the detailed metrics beyond just reward reveals distinct behavioral patterns across different Domain Randomization (DR) configurations, highlighting the importance of proper tuning.

Model	Timeout %	Time-to-fail	Min torso z	Angle std	Near-falls	Fwd vel
No DR	0%	366	0.687	0.035	3.3	2.68
DR all $\pm 20\%$	100%	3000	0.978	0.020	0.0	2.67
DR all $\pm 40\%$	0%	464	0.686	0.062	3.0	2.78

NO DR

Policies trained without DR exhibit **early failure** and operate with a **small stability margin**, leading to frequent near-fall events due to environmental mismatches.

MODERATE DR ($\Delta = 0.2$)

This optimal configuration yields **no failures**, resulting in **smooth and stable motion**. The policies demonstrate a **large safety margin**, adapting well to target environments.

EXCESSIVE DR ($\Delta = 0.4$)

Overly strong DR induces **large oscillations** in robot behavior, eventually leading to **delayed but unavoidable failure** as the policy struggles to converge effectively.

Analysis of Latent Representations

The goal is to understand how the policy internally represents states under different training strategies, especially in relation to domain randomization.



Activation Extraction

We analyze the activations of the penultimate layer of the policy network, which encodes crucial information for decision-making and action selection.



Dimensionality Reduction

We use methods such as **PCA** and **t-SNE** to visualize the high-dimensional structure of latent representations in a more manageable space.



Global Structure (PCA)

PCA allows us to determine whether states from source and target domains cluster in distinct or overlapping regions, revealing the **separability** or **mixing** of domains.



Local Relationships (t-SNE)

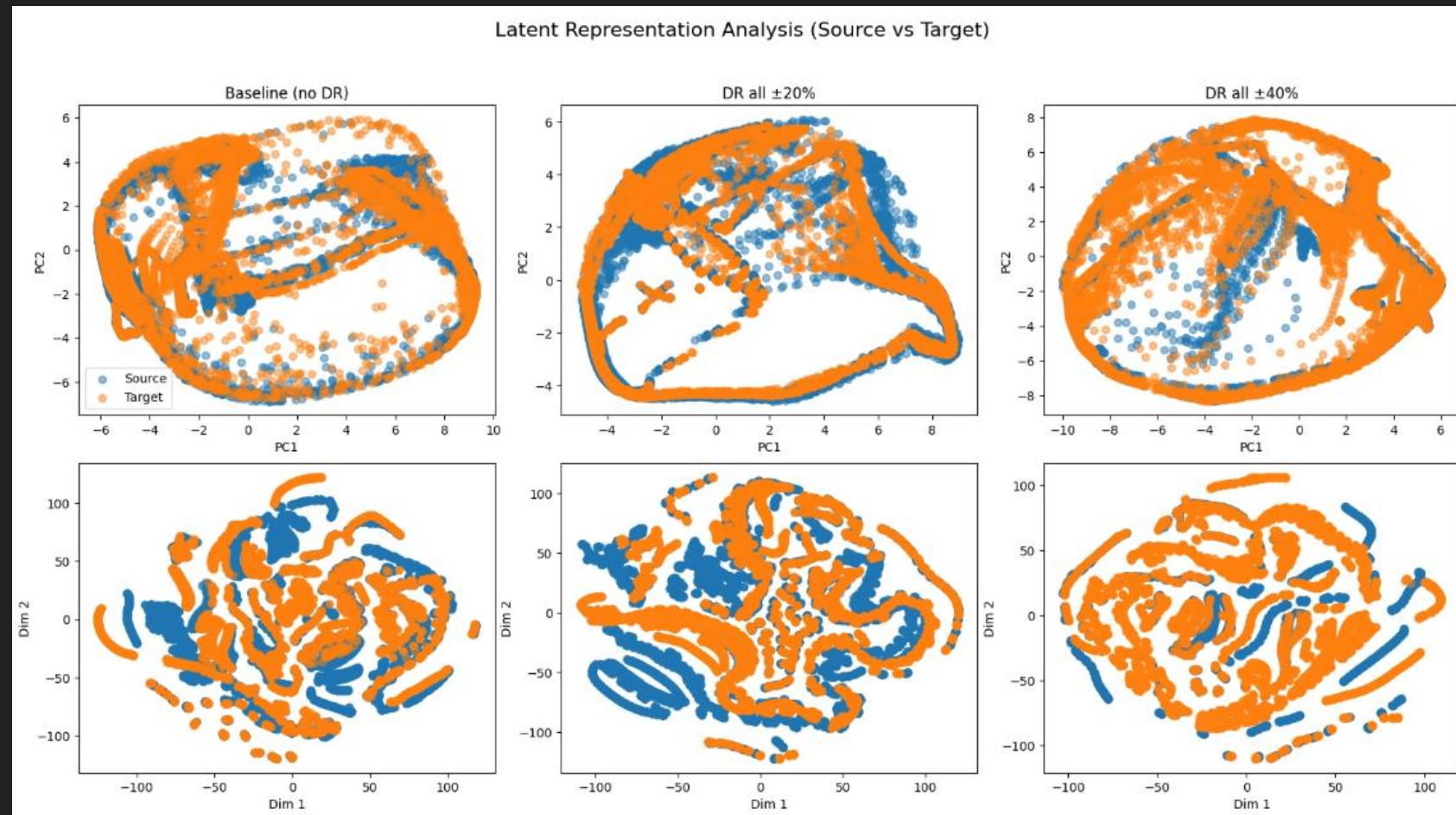
t-SNE focuses on neighborhood relationships, showing whether conceptually similar states are encoded **similarly** by the policy, regardless of domain.



Domain Invariance

By comparing these representations, we evaluate whether Domain Randomization (DR) effectively induces internal representations that are **invariant** to the domain.

Analysis of Latent Representations: Key Interpretations



Latent Representation Analysis: Comparison of source (orange) and target (blue) domain distributions across different DR configurations.

👤 BASELINE (NO DR)

Policies trained without Domain Randomization encode source and target environments **distinctly**, leading to domain-dependent representations and limited transferability.

⚙️ MODERATE DR

Moderate DR fosters **aligned and structured** latent representations, indicating domain-invariant behavior and enabling effective sim-to-sim transfer.

👤 EXCESSIVE DR

Excessive DR results in **mixed but unstructured** representations, causing policies to struggle with generalization and stability despite attempting domain mixing.

Final Conclusions and Key Takeaways

The study of Domain Randomization (DR) reveals that it is an indispensable tool for improving the transfer of agent policies from simulation to real environments, provided it is applied with the right strategy.



Improves Transfer

Domain Randomization **significantly optimizes** the transfer of policies from sim-to-sim, closing the gap between simulations and reality.



Robust Policies

The resulting policies operate with **wider stability margins**, exhibit smoother behavior, and learn internal representations that are better aligned.

These findings underscore the importance of careful configuration of Domain Randomization to achieve truly robust and transferable reinforcement learning systems.



Moderate Randomization

A **moderate level of randomization** is crucial for stability and performance, avoiding both insufficient and excessive variability.



Beyond Reward

Deep analysis of metrics beyond reward explains **why DR works**, revealing its effects on stability and latent representations.